

React HandsOn 13:-

Explain Various Ways of Conditional Rendering

Conditional Rendering means displaying different UI based on a condition.

React supports several methods.

Method 1: Using if...else

```
function App() {  
  const isLoggedIn = true;  
  if (isLoggedIn) {  
    return <h1>Welcome User</h1>;  
  } else {  
    return <h1>Please Login</h1>;  
  }  
}
```

Output

Welcome User

Method 2: Using Element Variables

Store the JSX element in a variable.

```
function App() {  
  const isLoggedIn = false;  
  let message;  
  if (isLoggedIn)  
    message = <h1>Welcome</h1>;  
  else  
    message = <h1>Please Login</h1>;  
  return (  
    <div>  
      {message}  
    </div>  
  );  
}
```

```
}
```

Method 3: Using Ternary Operator

```
function App() {  
  const isLoggedIn = true;  
  return (  
    <h1>  
      {isLoggedIn ? "Welcome" : "Login"}  
    </h1>  
  );  
}
```

Method 4: Using Logical AND (&&)

```
function App() {  
  const isAdmin = true;  
  return (  
    <div>  
      {isAdmin && <h2>Admin Panel</h2>}  
    </div>  
  );  
}
```

If isAdmin is false, nothing is displayed.

Method 5: Returning null

```
function App() {  
  const show = false;  
  if (!show)  
    return null;  
  return <h1>Hello</h1>;  
}
```

No output is rendered.

Explain how to render multiple components

React applications are built by combining multiple components.

Example

Header.js

```
function Header() {  
  return <h1>React Application</h1>;  
}  
  
export default Header;
```

Footer.js

```
function Footer() {  
  return <h3>Copyright 2026</h3>;  
}  
  
export default Footer;
```

App.js

```
import Header from "./Header";  
import Footer from "./Footer";  
  
function App() {  
  return (  
    <div>  
      <Header />  
      <p>Welcome to React</p>  
      <Footer />  
    </div>  
  );  
}  
  
export default App;
```

Output

React Application

Welcome to React

Rendering Multiple Components in One Component

```
function Student() {  
  return <h2>Student Component</h2>;  
}  
  
function Teacher() {  
  return <h2>Teacher Component</h2>;  
}  
  
function App() {  
  return (  
    <div>  
      <Student />  
      <Teacher />  
    </div>  
  );  
}
```

Define List Component

A List Component displays a collection of items by looping through an array.

React generally uses the `map()` function to render lists.

Example

```
function App() {  
  const fruits = ["Apple", "Orange", "Mango"];  
  return (  
    <ul>  
      {fruits.map((fruit) => (  
        <li>{fruit}</li>  
      ))}  
    </ul>  
  );  
}
```

Output

Apple

Orange

Mango

List of Objects

```
const students = [  
  {id:1,name:"Arpita"},  
  {id:2,name:"Rahul"},  
  {id:3,name:"Anjali"}  
];  
  
<ul>  
  
  {  
    students.map(student=>(  
      <li key={student.id}>  
        {student.name}  
      </li>  
    ))  
  }  
</ul>
```

Explain About Keys in React Applications

A Key is a unique identifier given to elements in a list.

Keys help React identify which items have changed, been added, or removed.

Syntax

```
key={uniqueValue}
```

Example

```
const numbers=[10,20,30];  
  
<ul>  
  
  {  
    numbers.map((num,index)=>(  

```

```
<li key={index}>{num}</li>
))
}
</ul>
```

Better Example

```
const employees=[
  {id:101,name:"John"},
  {id:102,name:"David"}
];
employees.map(emp=>
  <li key={emp.id}>
    {emp.name}
  </li>
)
```

Use a unique ID whenever possible instead of the array index.

- Improve performance.
- Help React update only changed elements.
- Prevent rendering errors.
- Required when rendering lists.

Explain How to Extract Components with Keys

When a list becomes large, create a separate component for each item.

Student.js

```
function Student(props){
  return(
    <li>
      {props.name}
    </li>
  )
}
```


);

}

export default Student;

App.js

import Student from "./Student";

const students=[

{id:1,name:"Arpita"},

{id:2,name:"Rahul"},

{id:3,name:"Anjali"}]

);

function App(){

return(

{

students.map(student=>

<Student

key={student.id}

name={student.name}

/>

)

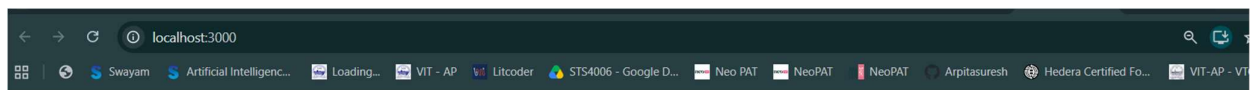
}

);

}

export default App;

OUTPUT:



Course Details

Angular

4/5/2021

React

6/3/2021

Book Details

Master React

670

Deep Dive into Angular 11

800

Mongo Essentials

450

Blog Details

React Learning

Stephen Biz

Welcome to learning React!

Installation

Schwezenier

You can install React from npm.